Priority Queues and Heaps

[CSE2010] Data Structures

Department of Data Science

Priority Queue

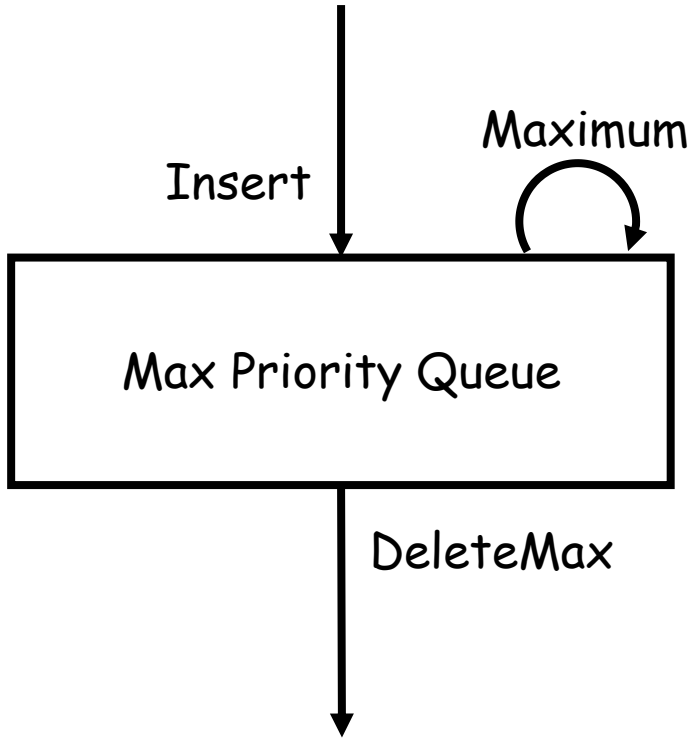
A collection of prioritized elements that allows arbitrary element insertion, and allows the removal of the element with the first priority

Priority Queues

- Abstract data type for maintaining a set S of elements, each with an associated value called key.
- Max priority queue supports
  - Insert(S, x): insert the element x into the set S, which is equivalent to the operation $S = S \cup \{x\}$.
  - Maximum(S): return the element of S with the largest key
  - DeleteMax(S): remove and return the element of S with the largest key

Maximum Priority Queues

- Operations
  - Insert(S, x)
  - Maximum(S)
  - DeleteMax(S)
- Difference from queues: in priority queues, the order in which the elements enter the queue may not be the same in which they were processed.



Applications

- Hospital emergency system
  - Suppose we are given patients A, B, and C.
  - What if a new patient D with severity 3 just comes over now?

Front of priority queue →

Patient C – Severity: 10
Patient A – Severity: 6
Patient B – Severity: 2

Patient C – Severity: 10
Patient A – Severity: 6
Patient D – Severity: 3
Patient B – Severity: 2

Applications

- Job scheduling
- Data compression: Huffman Coding algorithm
- Shortest path problem: Dijkstra’s algorithm
- Minimum spanning tree: Prim’s algorithm
- Event-driven simulation: customers in a line
- Selection problem: finding kth smallest element

How can we implement a priority queue?

Unordered Array

Ordered Array

Binary Heap

⋮

Binary Heap

- Binary tree that stores a collection of items and satisfies 2 conditions

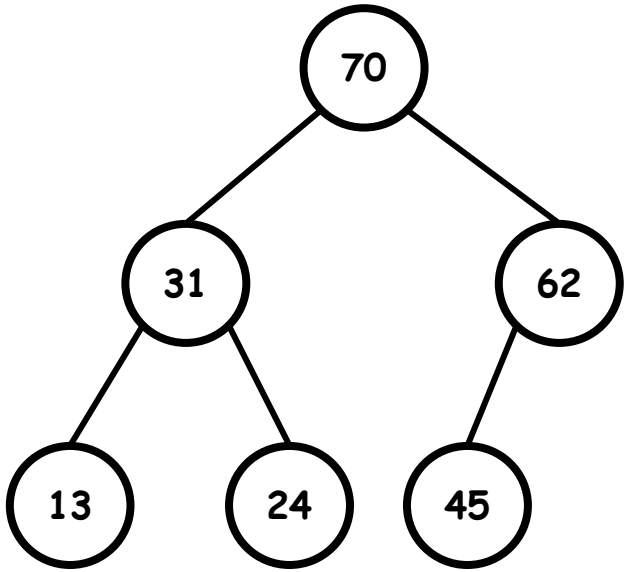
1. The value of each node ≥ (≤) the values of its children (heap property).
   - Max heap, min heap
2. Completely filled with nodes; no nodes are missing (complete tree)

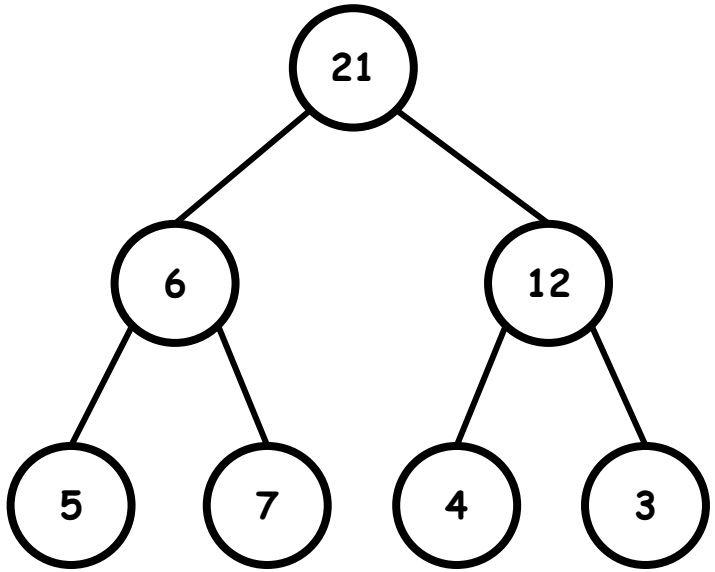
- Time complexity

Operation	Time complexity
Insertion	O(logn)
DeleteMax	O(logn)
Maximum	O(1)

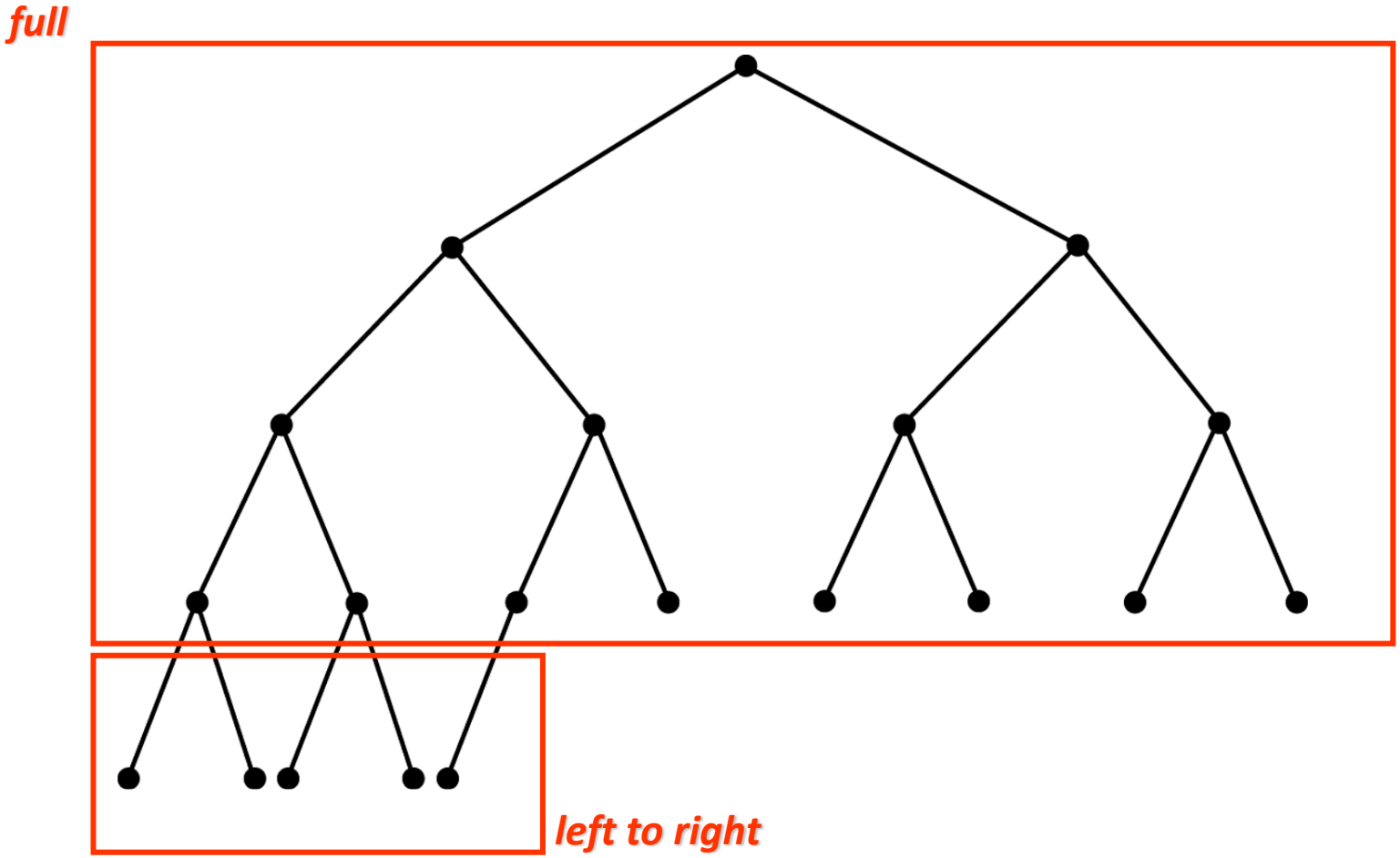
Heap Property

- The value of each node ≥ the values of its children
- Are they max heaps?



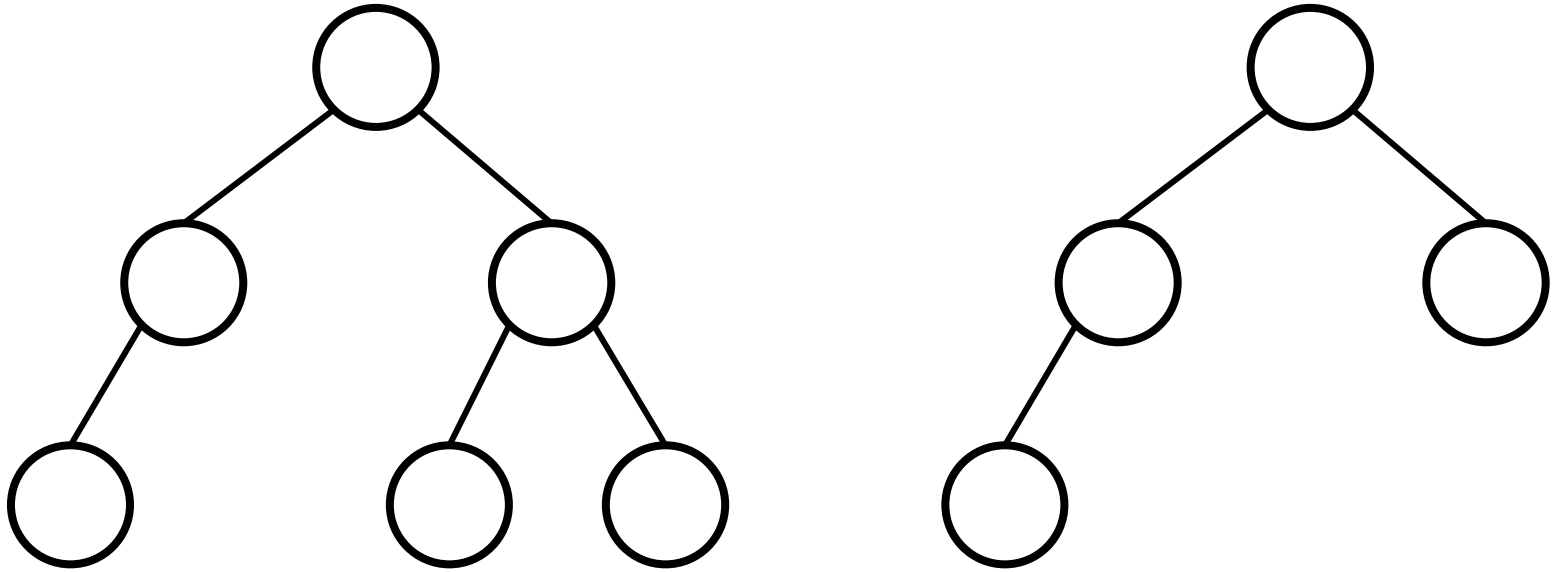


Complete Binary Trees



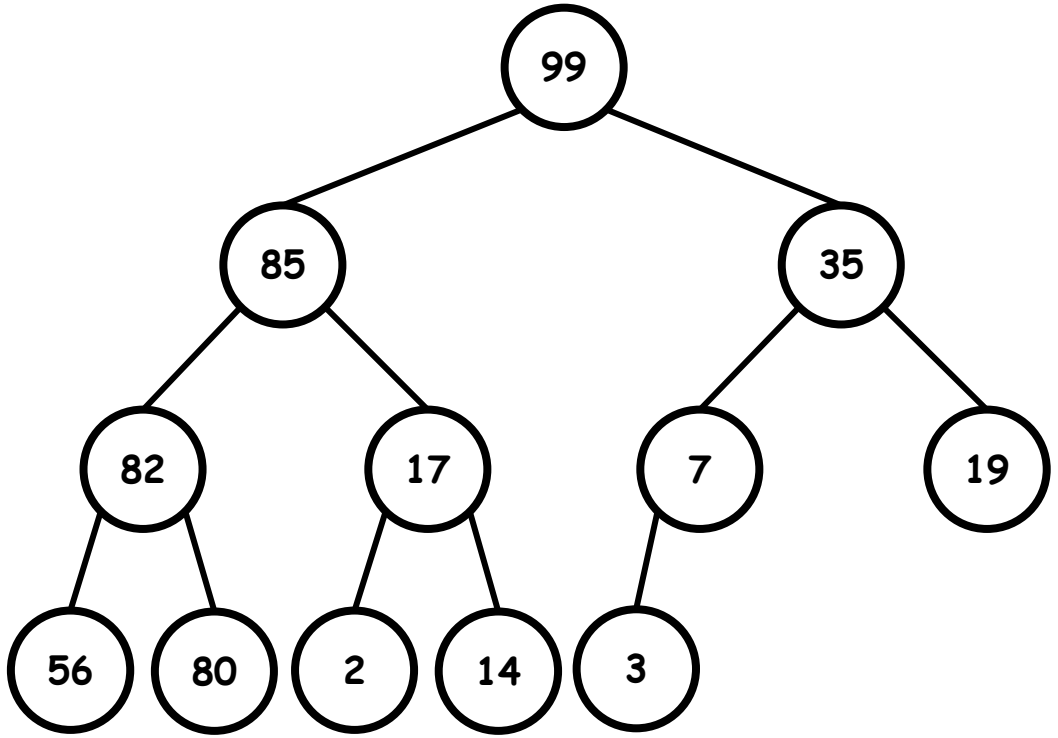
Complete Trees

- Heap with height h is a complete binary tree
  - if levels $0,1,2,\ldots,h-1$ have the maximum number of nodes possible (level i has $2i$ nodes, for $0 \leq i \leq h-1$) and the remaining nodes at level h reside in the **leftmost** possible positions at that level.
- Are they complete?



No Search in Heaps

- While the heap condition dictates that the heap be ordered a certain way, this ordering is still useless when it comes to searching for a value within the heap.

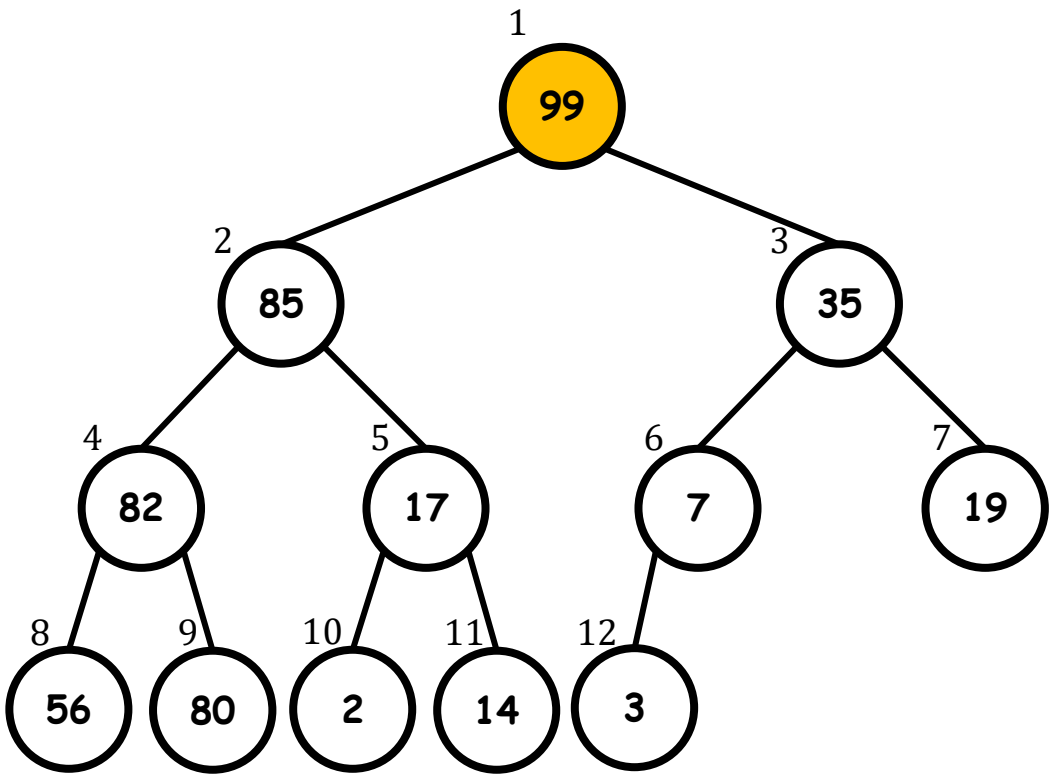


Arrays as Heaps

- Binary heap with n elements can be stored in array since it is a complete tree.

- First element is always the maximum.

1	2	3	4	5	6	7	8	9	10	11	12
99	85	35	82	17	7	19	56	80	2	14	3



Declaration of Heap

```
class Heap {
public:
  int* array;
  int count; // Number of elements
  int capacity; // Size of the heap
  Heap(int capacity);// Constructor
  // ⋮
};
```

- C/C++: array start from 0.

```
Heap::Heap(int cap) {
  count = 0;
  capacity = cap;
  array = new int[cap];
}
```

Arrays as Heaps

Parent(i)

return $\lfloor i/2 \rfloor$

Left(i)

return $2i$

Right(i)

return $2i + 1$

- Pseudocode: array starts from 1

```
int Parent(int i) {
  return (i - 1) / 2;
}

int Left(int i) {
  return 2 * i + 1;
}

int Right(int i) {
  return 2 * i + 2;
}
```

- C/C++: array starts from 0.

Maximum

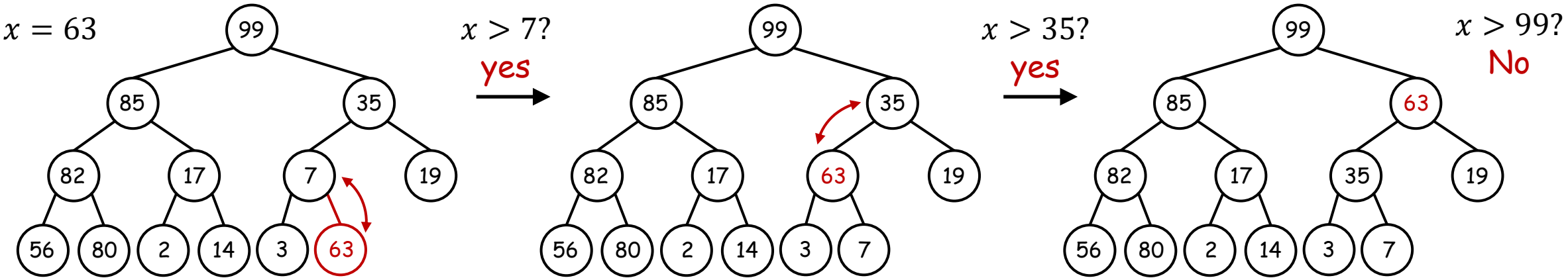
Maximum(A)

  return A[1]

```
int Maximum(Heap& h) {
  // h.count is the number of elements
  if (h.count == 0) {
    cerr<<"no elements"<<endl;
    exit(1);
  }
  return h.array[0];
}
```

Insertion of 63

1. Create a node containing a new value x, and insert the node at the next available rightmost spot in the bottom level.
   - This node becomes the heap's last node.
2. If x > the value of its parent node, swap these two values.
3. Repeat Step 2, moving x up through the heap, until the node with x has a parent node whose value > x.



Inserting

Insert(A, key)

i = A.heapSize

A.heapSize = A.heapSize + 1

A[heapSize] = key

while i > 1 and A[Parent(i)] < key

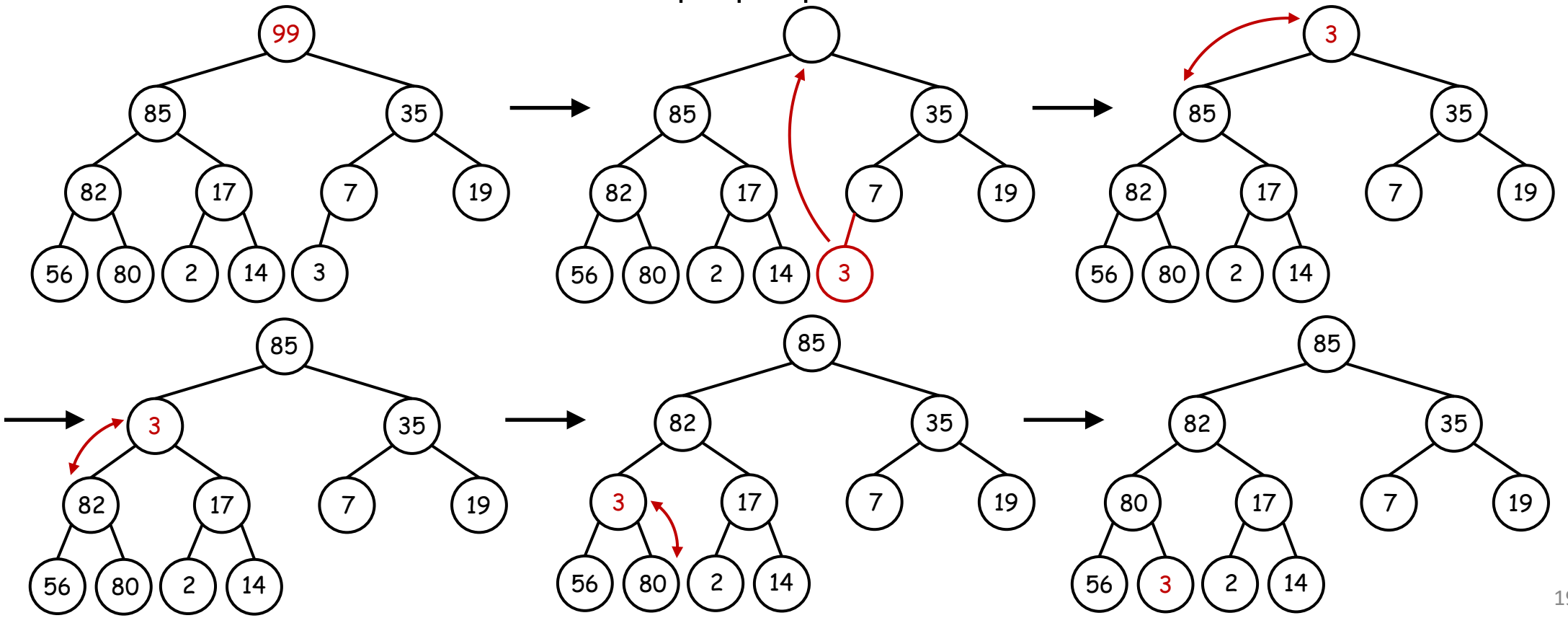
exchange A[i] with A[Parent(i)]

i = Parent(i)

```
void Insert(Heap& h, int key) {
  int i = h.count;
  int p = Parent(i);
  while (i > 0 && h.array[p] < key) {
    h.array[i] = h.array[p];
    i = p;
    p = Parent(i);
  }
  h.array[i] = key;
  ++h.count;
}
```

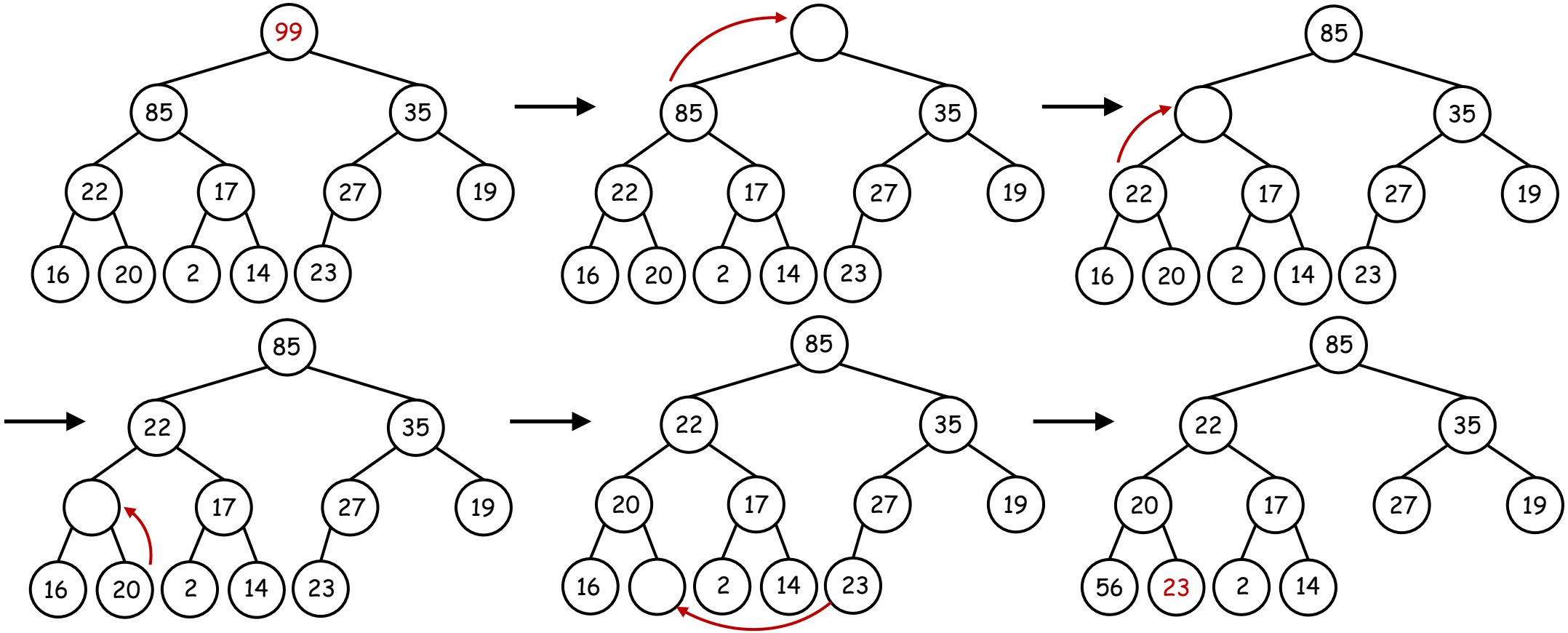
Deleting Maximum

1. Move the last node into where the root node was, effectively removing the original root node.
2. Trickle the root node down into its proper place.



Deleting Maximum: What If?

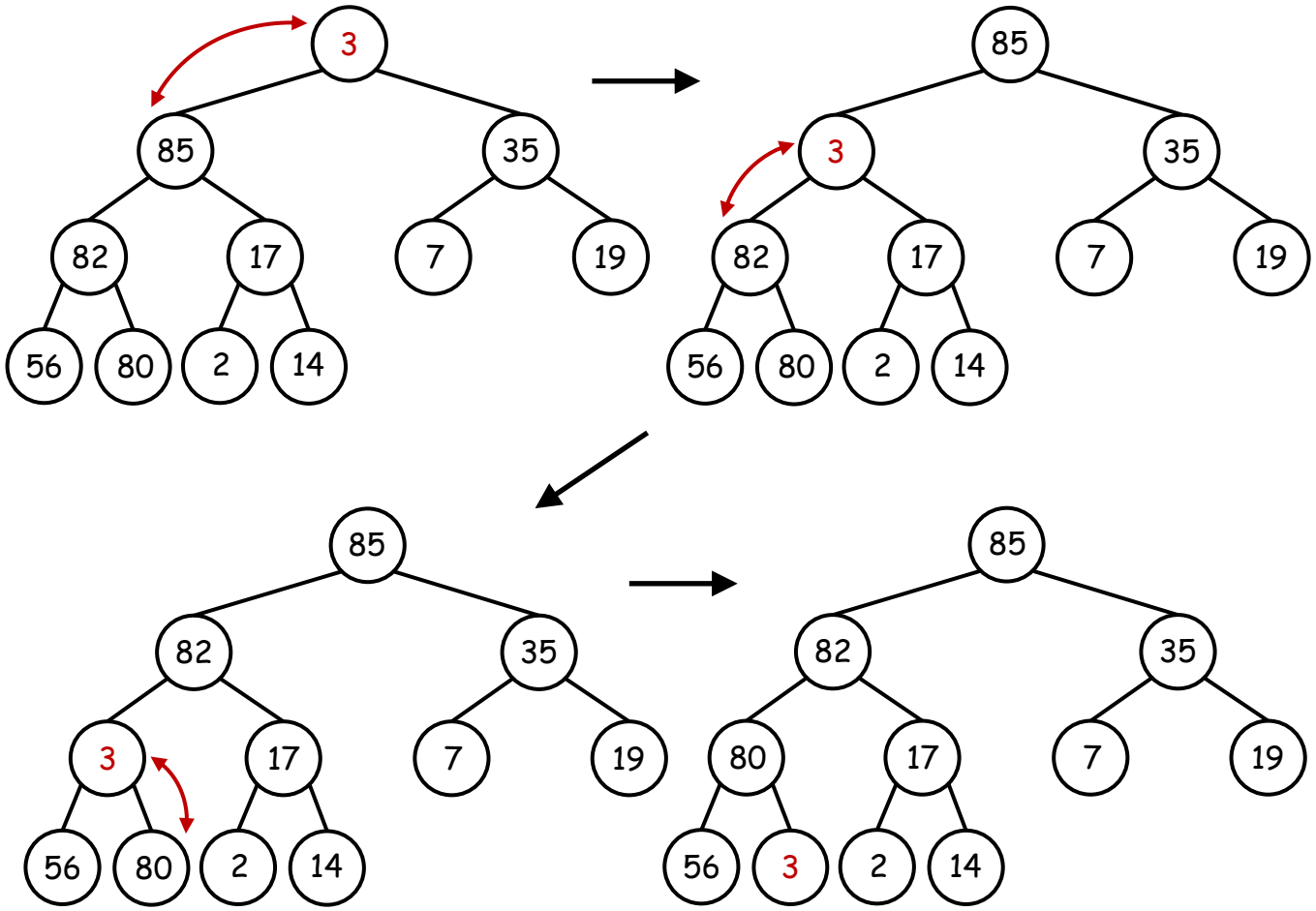
- Why do we move the last node to the root node?
  - What if we move the child with the larger value upward, and then move the last node to the empty node in the same level?



Maintaining Heap Property

- Heapify (Percolate Down)

1. Check both children of the current node and see which one is higher.
2. If the current node is **smaller than** the larger of the two children, **swap** the current node with the larger child.
3. Repeat Steps 1 and 2 until the current node has no children who are **greater than** it.



MaxHeapify

MaxHeapify(A, i)

l = Left(i)

r = Right(i)

if l ≤ A.heapSize and A[l] > A[i]

largest = l

else

largest = i

if r ≤ A.heapSize and A[r] > A[largest]

largest = r

Check if the current node is smaller than the large of the two children

if largest ≠ i

exchange A[i] with A[largest]

MaxHeapify(A, largest)

Swap the current node with the larger child

```
void MaxHeapify(Heap& h, int i) {
  int l, r, largest, temp;
  l = Left(i);
  r = Right(i);
  if (l != -1 && h.array[l] > h.array[i])
    largest = l;
  else
    largest = i;
  if (r!=-1 && h.array[r]>h.array[largest])
    largest = r;
  if (largest != i) {
    temp = h.array[i];
    h.array[i] = h.array[largest];
    h.array[largest] = temp;
    MaxHeapify(h, largest);
  }
}
```

Deleting Maximum

DeleteMax(A)

if A.heapSize < 1

error "heap underflow"

max = A[1]

A[1] = A[A.heapSize]

A.heapSize =A.heapSize – 1

MaxHeapify(A, 1)

return max

Replace the max with the last element

Shrink the size of A by 1

Find the right position for the updated root element

```
int DeleteMax(Heap& h) {
  if (h.count == 0) {
    cerr<<"heap underflow"<<endl;
    exit(1);
  }
  int largest = h.array[0];
  h.array[0] = h.array[h.count - 1];
  --h.count;
  MaxHeapify(h, 0);
  return largest;
}
```

Heapsort

Heapsort(A)

BuildMaxHeap(A)

for i = A.length down to 2

exchange A[1] and A[i] — Delete the max, and move it to the last position of A

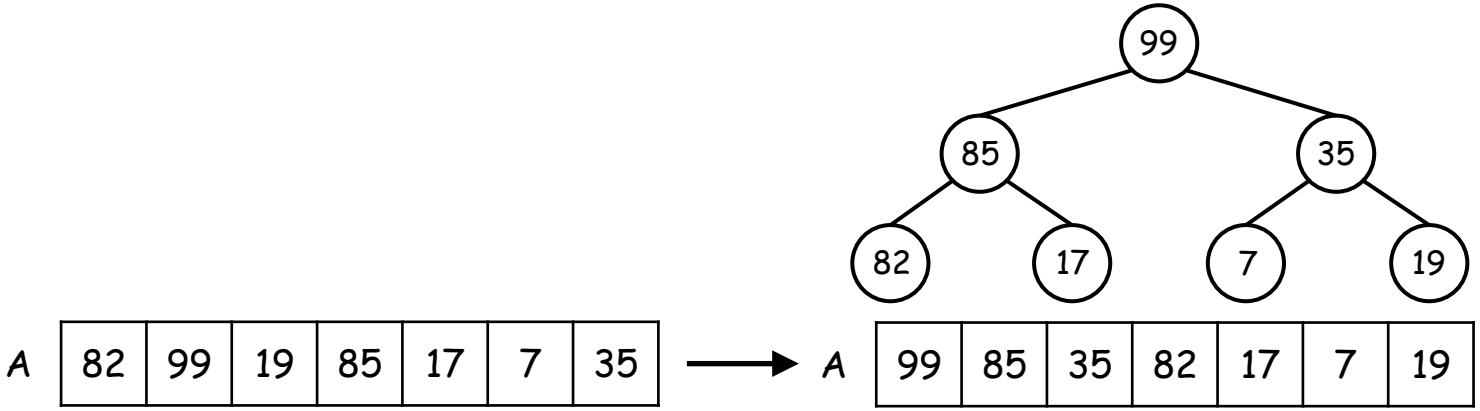
A.heapSize = A.heapSize – 1 — Shrink the size of A by 1

MaxHeapify(A, 1) — Find the right position for the updated root element

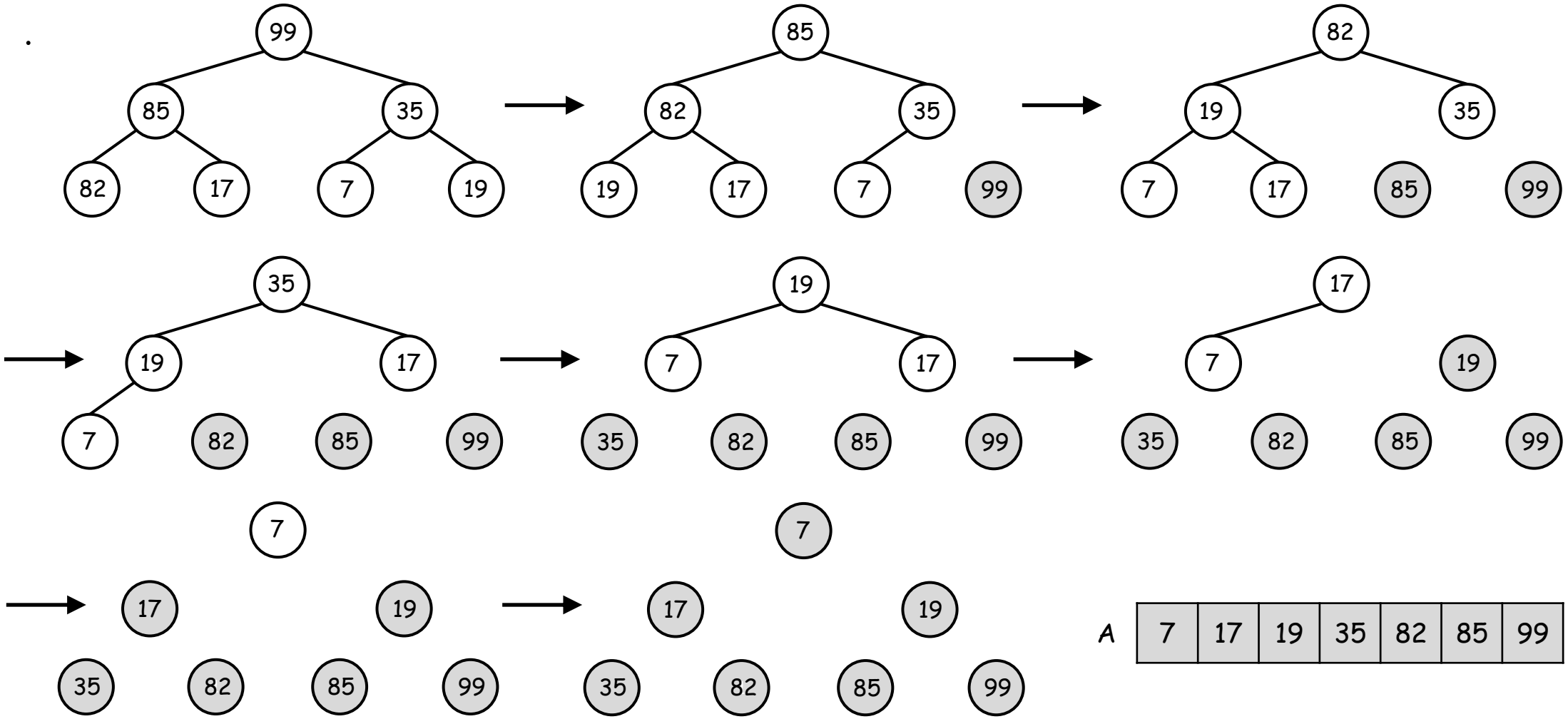
```
void Heapsort(int A[], int n){
  Heap h(n);
  BuildMaxHeap(h, A, n);
  int oldSize = h.count;
  for (int i = n - 1; i > 0; --i) {
    // h.array[0]: the largest
    int temp = h.array[0];
    h.array[0] = h.array[h.count - 1];
    h.array[0] = temp;
    --h.count;
    MaxHeapify(h, 0);
  }
  h.count = oldSize;
}
```

Heapsort

- Given array A, build a max heap.



Heapsort



A						
7	17	19	35	82	85	99

Heapsort

- Build a binary heap of n elements: $O(n)$
- DeleteMax operation n times: $O(nlogn)$
- Need extra space to save the sorted list?
  - Use the last element in the previous heap
  - Use only a small amount of memory in addition to the sequence storing the objects to be sorted (**in-place**)

Building Max Heap

- Build a heap containing n keys takes $O(nlogn)$ with consecutive insertions.
- However, building a heap can take $O(n)$ if keys are already in array.
- Starting with the **lowest non-leaf node**, working back towards root, perform percolating-down (heapifying) on each node of the tree.

BuildMaxHeap

BuildMaxHeap(A)

$A.heapSize = A.length$

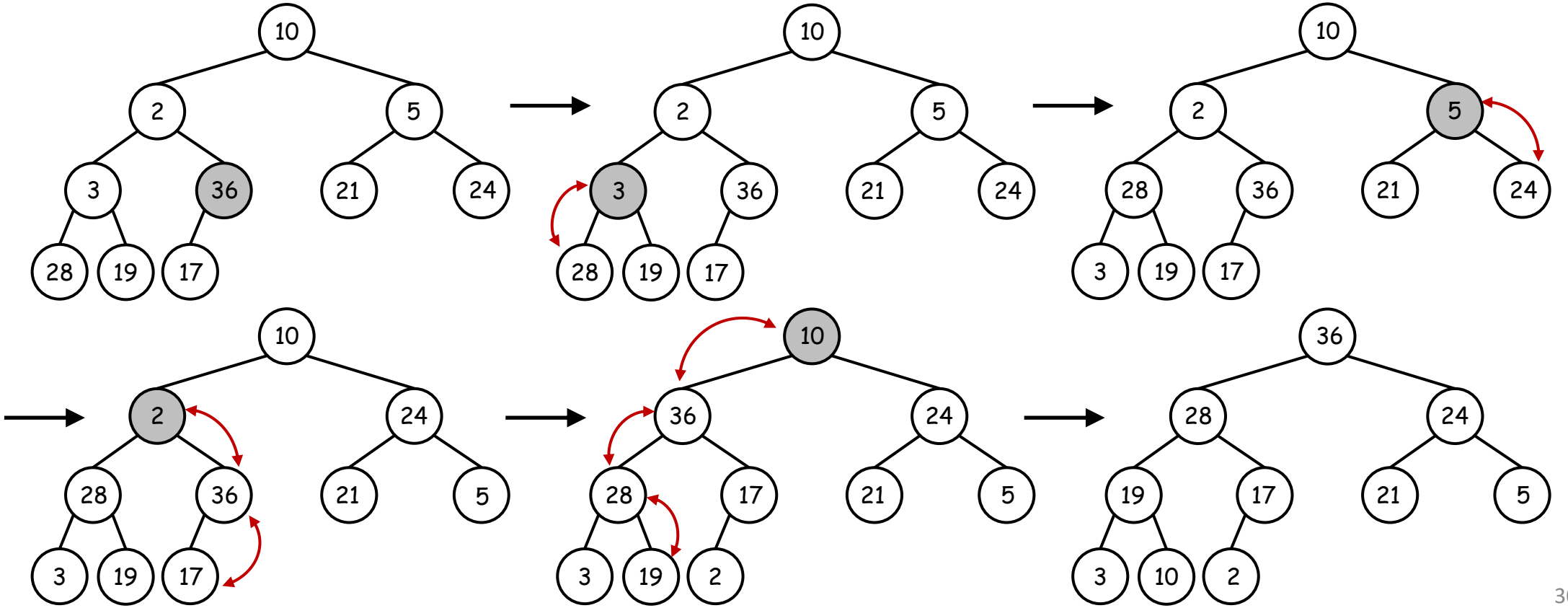
for i = $\lfloor A.length/2 \rfloor$ down to 1

MaxHeapify(A, i)

```
void BuildMaxHeap(Heap& h, int A[], int n){
  for (int i = 0; i < n; i++)
    h.array[i] = A[i];
  h.count = n;
  for (int i = (n-1)/2; i >= 0; i--)
    MaxHeapify(h, i);
}
```

Building Max Heap

A									
10	2	5	3	36	21	24	28	19	17



Why Does Building a Heap Take O(n) Time?

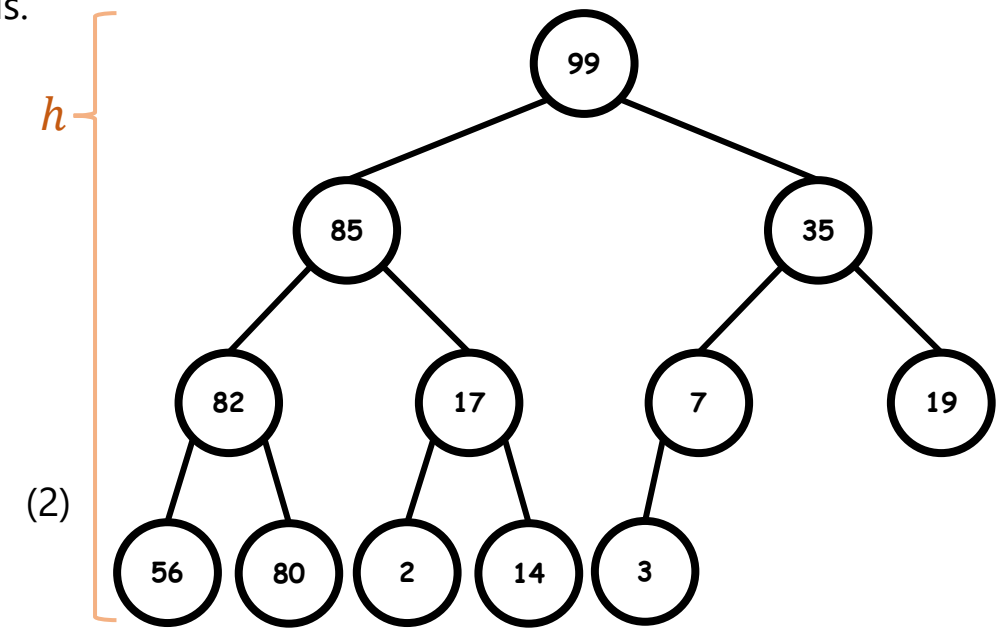
- A complete binary tree with height h has $2^h < n \leq 2^{h+1} - 1$ nodes … (1)
- The complete binary tree has
  - One node at level 0, which may be percolated down to h levels.
  - Two nodes at level 1, which may be percolated down to h-1 levels.
  - Four nodes at level 2, which may be percolated down to h-2 levels.

  ⋮
- Total number S of computations
  - $S \ = 1 \cdot h + 2 \cdot (h-1) + 4 \cdot (h-2) + \cdots + 2^{h-2} \cdot 2 + 2^{h-1} \cdot 1$
  - $2S = 2 \cdot h + 4 \cdot (h-1) + 8 \cdot (h-2) + \cdots + 2^{h-1} \cdot 2 + 2^h \cdot 1$
  - $2S - S = -h + \left(2 + 4 + \cdots + 2^{h-1}\right) + 2^h$

    $= -h + \left(1 + 2 + 4 + \cdots + 2^{h-1}\right) + 2^h - 1$

    $= -h + 2 \cdot \left(2^h - 1\right)$ $\quad 2^h < n$

    $S \ < 2n - h - 2 \leq 2n$ (3)



(2)

Advantages of Priority Queues & Heaps

Operation	Worst Case
Maximum	O(1)
Insert	O(logn)
DeleteMax	O(logn)
Heapsort	O(nlogn)

- Primary function of a priority queue is to allow us immediate access to the **highest-priority item** in the queue.
  - Ordered array requires slower insertions of O(n) to ensure that each new value is in its proper place.
- **Heaps** are the tool for building priority queues.